ERFA — Guide d'explication complet

Ligne par ligne — Chaque cellule du notebook

Cellule 1 — Imports et chargement du dataset

Rôle de cette cellule

Importer toutes les bibliothèques nécessaires et charger le fichier CSV du dataset bancaire. C'est le point d'entrée de tout le notebook.

Code complet

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

df = pd.read_csv('Churn_Modelling.csv')
print('Shape du dataset :', df.shape)
print(df.head())
print(df.info())
```

Explication ligne par ligne

`import pandas as pd`	Pandas = bibliothèque de manipulation de données. 'pd' est l'alias standard. Permet de lire CSV, créer tableaux, filtrer colonnes.
`import numpy as np`	NumPy = calcul numérique (matrices, statistiques). Utilisé pour les calculs Gini et les arrays sklearn.
`import matplotlib.pyplot as plt`	Matplotlib = bibliothèque de graphiques de base. Utilisée pour tracer les histogrammes et courbes ROC.
`import seaborn as sns`	Seaborn = graphiques statistiques avancés construits sur matplotlib. Utilisé pour countplot de la distribution des classes.
`import warnings`	Module Python standard pour gérer les avertissements.
`warnings.filterwarnings('ignore')`	Supprime les warnings sklearn (ConvergenceWarning de MLPClassifier etc.) pour un output propre.
`df = pd.read_csv('Churn_Modelling.csv')`	Charge le fichier CSV dans un DataFrame pandas 'df'. df devient le tableau principal avec 10 000 lignes x 14 colonnes.
`print('Shape :', df.shape)`	Affiche (10000, 14) — 10 000 clients, 14 colonnes. Permet de vérifier que le fichier est bien chargé.
`print(df.head())`	Affiche les 5 premières lignes pour voir la structure des données et les noms des colonnes.
`print(df.info())`	Affiche les types de données de chaque colonne et le nombre de valeurs non-nulles. Permet de détecter les valeurs manquantes.

Cellule 2 — Nettoyage des données

Rôle de cette cellule

Vérifier l'absence de valeurs manquantes et supprimer les colonnes inutiles qui n'apportent aucune information prédictive. L'article précise : 'not including unnecessary attributes like customer ID, and customer name'.

Code complet

```
print('Valeurs manquantes par colonne :')
print(df.isnull().sum())
print()

df = df.drop(columns=['RowNumber', 'CustomerId', 'Surname'])

print('Colonnes après nettoyage :', df.columns.tolist())
print('Nouvelle shape :', df.shape)
```

Explication ligne par ligne

`df.isnull().sum()`	Compte le nombre de valeurs NaN (vides) dans chaque colonne. Si tout est 0 : aucune valeur manquante, pas besoin d'imputation.
`df.drop(columns=['RowNumber',...])`	Supprime les 3 colonnes inutiles : RowNumber (numéro de ligne), CustomerId (identifiant unique), Surname (nom de famille). Ces colonnes n'ont aucun pouvoir prédictif sur le churn.
`df = df.drop(...)`	Le résultat est réassigné à df. Sans le 'df =', le drop ne serait pas sauvegardé.
`df.columns.tolist()`	Affiche la liste des colonnes restantes pour vérifier que la suppression s'est bien passée.

Cellule 3 — Encodage des variables catégorielles

Rôle de cette cellule

Convertir les colonnes texte (Gender, Geography) en valeurs numériques. Les algorithmes ML ne peuvent pas traiter directement les chaînes de caractères. L'article dit : 'solving the problem of categorical attributes and converting it to single numerical values'.

Code complet

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

# Encodage Gender : Female=0, Male=1
df['Gender'] = le.fit_transform(df['Gender'])

# Encodage Geography : One-Hot Encoding
df = pd.get_dummies(df, columns=['Geography'], drop_first=True)

print('Colonnes après encodage :', df.columns.tolist())
print('Shape :', df.shape)
print(df.head())
```

Explication ligne par ligne

`from sklearn.preprocessing import LabelEncoder`	Importe LabelEncoder depuis sklearn. Cet outil transforme des catégories textuelles en nombres entiers.
`le = LabelEncoder()`	Crée une instance du LabelEncoder. C'est un objet réutilisable qui 'apprend' le mapping texte -> nombre.
`df['Gender'] = le.fit_transform(df['Gender'])`	fit_transform : apprend le mapping (Female=0, Male=1) puis l'applique. Remplace la colonne Gender textuelle par des 0/1.
`pd.get_dummies(df, columns=['Geography'], drop_first=True)`	One-Hot Encoding sur Geography. Crée 2 nouvelles colonnes binaires : Geography_Germany et Geography_Spain. drop_first=True évite la multicolinéarité en supprimant Geography_France (redondant).
`df = pd.get_dummies(...)`	Le DataFrame df est remplacé par la version avec les nouvelles colonnes Geography_Germany et Geography_Spain.

Cellule 4 — Normalisation (StandardScaler)

Rôle de cette cellule

Mettre toutes les features à la même échelle pour éviter qu'une variable avec de grandes valeurs (Balance = 200 000) domine les variables avec de petites valeurs (NumOfProducts = 1-4). L'article dit : 'the large values must be made between -2 and +2'.

Code complet

```
from sklearn.preprocessing import StandardScaler

# Séparation features / cible
features = df.drop('Exited', axis=1)
target   = df['Exited']

# Normalisation — ramène les valeurs typiquement entre -2 et +2
scaler   = StandardScaler()
X_scaled = scaler.fit_transform(features)

X = pd.DataFrame(X_scaled, columns=features.columns)
y = target

print('Features normalisées — statistiques :')
print(pd.DataFrame(X_scaled, columns=features.columns).describe().round(2))
```

Explication ligne par ligne

`features = df.drop('Exited', axis=1)`	Sépare les variables indépendantes (features) de la variable cible. axis=1 signifie supprimer une colonne (vs axis=0 pour une ligne).
`target = df['Exited']`	Extrait la colonne cible 'Exited' (0=retain, 1=churn). C'est le y que le modèle devra prédire.
`scaler = StandardScaler()`	Crée le scaler. StandardScaler calcule la moyenne et l'écart-type de chaque colonne pour normaliser.

`X_scaled = scaler.fit_transform(features)`	fit : calcule moyenne/écart-type. transform : applique la formule z = (x - moyenne) / écart-type. Résultat : valeurs centrées autour de 0, typiquement entre -2 et +2.
`X = pd.DataFrame(X_scaled, columns=features.columns)`	Recrée un DataFrame avec les noms de colonnes originaux. X_scaled est un array numpy, on le reconvertit en DataFrame pour conserver les noms.
`y = target`	y est la variable cible. Convention standard ML : X = features, y = cible.

Cellule 5 — Sélection des features (corrélation >= 10%)

Rôle de cette cellule

Identifier les variables les plus importantes et éliminer celles qui n'ont pas de relation significative avec le churn. L'article précise : 'The only rest features selected will be included in the model if the correlation of fs and Y >= 10%'. Correspond à la Figure 2 du papier.

Code complet

```
correlations = df.corr(numeric_only=True)['Exited'].abs().sort_values(ascending=False)

print('Corrélations avec Exited :')
print(correlations)

# Visualisation — Figure 2 du papier
plt.figure(figsize=(10, 6))
correlations.drop('Exited').plot(kind='bar', color='steelblue')
plt.axhline(y=0.10, color='red', linestyle='--', label='Seuil 10%')
plt.title('Top Predictive Features — Figure 2 du papier')
plt.ylabel('Importance (corrélation absolue)')
plt.xlabel('Features')
plt.legend()
plt.tight_layout()
plt.show()

# Sélection des features avec corrélation >= 10%
selected_features = correlations[correlations >= 0.10].index.tolist()
selected_features.remove('Exited')
print('Features sélectionnées :', selected_features)

X = X[selected_features]
```

Explication ligne par ligne

`df.corr(numeric_only=True)['Exited']`	Calcule la matrice de corrélation de Pearson entre toutes les colonnes numériques. On garde seulement la colonne 'Exited' pour voir la relation de chaque feature avec la cible.
`.abs()`	Prend la valeur absolue. Une corrélation négative forte (-0.35) est aussi importante qu'une positive (+0.35).
`.sort_values(ascending=False)`	Trié du plus corrélé au moins corrélé pour voir facilement les features les plus importantes.
`plt.axhline(y=0.10, color='red', linestyle='--')`	Trace une ligne horizontale rouge en pointillés à y=0.10 sur le graphique. Visualise le seuil de sélection du papier.

`correlations[correlations >= 0.10]`	Filtre : garde seulement les features dont la corrélation avec Exited est >= 10%. C'est le critère exact de l'article.
`.index.tolist()`	Récupère les noms des features (l'index du Series) et les convertit en liste Python.
`selected_features.remove('Exited')`	Retirer 'Exited' de la liste car c'est la variable cible, pas une feature.
`X = X[selected_features]`	Mise à jour X pour ne garder que les colonnes sélectionnées. C'est ce X réduit qui sera utilisé pour entraîner les modèles.

Cellule 6 — Vérification du déséquilibre (Gini)

Rôle de cette cellule

Mesurer quantitativement le déséquilibre des classes avec la formule Gini de l'article : Gini(DF) = 1 - sum(pi^2). Un dataset parfaitement équilibré a Gini = 0.5. Plus Gini est bas, plus le déséquilibre est grand.

Code complet

```
counts = y.value_counts()
p0 = counts[0] / len(y)   # probabilité classe retain
p1 = counts[1] / len(y)   # probabilité classe churn

gini = 1 - (p0**2 + p1**2)

print('Distribution des classes :')
print(f'  Retain (0) : {counts[0]} clients ({p0:.1%})')
print(f'  Churn  (1) : {counts[1]} clients ({p1:.1%})')
print(f'Gini imbalance : {gini:.4f}')

plt.figure(figsize=(5, 4))
sns.countplot(x=y, palette=['steelblue', 'coral'])
plt.title('Distribution des classes — Dataset original')
plt.show()
```

Explication ligne par ligne

`y.value_counts()`	Compte le nombre d'occurrences de chaque classe : {0: ~8000, 1: ~2000}. Montre la distribution de la variable cible.
`p0 = counts[0] / len(y)`	Calcule la probabilité de la classe 0 (retain). Exemple : 8000/10000 = 0.80 soit 80%.
`p1 = counts[1] / len(y)`	Calcule la probabilité de la classe 1 (churn). Exemple : 2000/10000 = 0.20 soit 20%.
`gini = 1 - (p0**2 + p1**2)`	Formule Gini exacte du papier. Avec p0=0.8 et p1=0.2 : gini = 1-(0.64+0.04) = 0.32. Valeur proche de 0 = fort déséquilibre. Valeur proche de 0.5 = équilibre parfait.
`sns.countplot(x=y, ...)`	Créer un diagramme à barres montrant visuellement la différence de taille entre les deux classes. Confirmer graphiquement le déséquilibre calculé.

Cellule 7 — Division Train/Test 80/20 (modèles de référence)

Rôle de cette cellule

Diviser le dataset en ensemble d'entraînement et de test pour les 4 modèles de référence. L'article utilise 80/20 pour comparer LR, SVM, NN, RF défaut.

Code complet

```
from sklearn.model_selection import train_test_split

# Division pour les 4 modèles de référence — 80/20
X_train_ref, X_test_ref, y_train_ref, y_test_ref = train_test_split(
    X, y, test_size=0.20, random_state=40
)
print('Train shape :', X_train_ref.shape)
print('Test  shape :', X_test_ref.shape)
```

Explication ligne par ligne

`from sklearn.model_selection import train_test_split`	Importe la fonction de division. C'est la fonction standard sklearn pour séparer données train/test.
`X_train_ref, X_test_ref, y_train_ref, y_test_ref`	4 variables créées simultanément : features d'entraînement, features de test, cibles d'entraînement, cibles de test. Le suffixe '_ref' rappelle que c'est pour les modèles de référence.
`test_size=0.20`	20% des données pour le test = 2000 clients. 80% pour l'entraînement = 8000 clients.
`random_state=40`	Fixe le générateur de nombres aléatoires. Avec random_state=40, la division sera toujours identique à chaque exécution. Valeur utilisée dans le papier.

Cellule 8 — Entraînement des 4 modèles défaut

Rôle de cette cellule

Entraîner et comparer les 4 algorithmes de l'article (LR, SVM, NN, RF) avec leurs paramètres par défaut pour identifier lequel est le meilleur avant l'amélioration ERFA. Résultats attendus : RF meilleur (~86%). Correspond à la Figure 3 et Table 1 du papier.

Code complet

```
from sklearn.linear_model    import LogisticRegression
from sklearn.svm             import SVC
from sklearn.neural_network  import MLPClassifier
from sklearn.ensemble        import RandomForestClassifier
from sklearn.metrics         import (accuracy_score, classification_report,
                                     confusion_matrix, roc_auc_score,
                                     RocCurveDisplay)

models = {
    'LR' : LogisticRegression(),
    'SVM': SVC(probability=True),
    'NN' : MLPClassifier(max_iter=1000),
    'RF' : RandomForestClassifier()
}
```

```
print('=== Résultats sur dataset DÉSÉQUILIBRÉ (80/20) ===')
results_imb = {}

for name, model in models.items():
    model.fit(X_train_ref, y_train_ref)
    acc = model.score(X_test_ref, y_test_ref)
    results_imb[name] = round(acc, 4)
    print(f'{name}: {acc:.4f}')
```

Explication ligne par ligne

`LogisticRegression()`	Régression Logistique avec paramètres sklearn par défaut. Modèle linéaire de classification. Simple mais efficace comme baseline.
`SVC(probability=True)`	Support Vector Machine. probability=True est obligatoire pour pouvoir calculer l'AUC-ROC (nécessite predict_proba). Sans ce paramètre, SVC n'expose pas les probabilités.
`MLPClassifier(max_iter=1000)`	Réseau de neurones (Multi-Layer Perceptron). max_iter=1000 augmente le nombre d'itérations pour assurer la convergence (défaut=200 trop faible).
`RandomForestClassifier()`	Forêt aléatoire avec paramètres défaut : 100 arbres, max_features='sqrt', max_depth=None. C'est le modèle de référence AVANT l'amélioration ERFA.
`models = { 'LR': ..., ... }`	Dictionnaire Python pour stocker les 4 modèles. Permet d'itérer sur tous avec une seule boucle au lieu de répéter le code 4 fois.
`results_imb = {}`	Dictionnaire vide qui va accumuler les accuracy de chaque modèle pour les visualiser ensuite.
`for name, model in models.items():`	Boucle sur les 4 modèles. .items() retourne des paires (clé, valeur) soit ('LR', LogisticRegression()), etc.
`model.fit(X_train_ref, y_train_ref)`	Entraîne le modèle sur les données d'entraînement. C'est ici que l'algorithme 'apprend' les patterns.
`model.score(X_test_ref, y_test_ref)`	Calcule l'accuracy sur le jeu de test. Retourne un float entre 0 et 1.
`results_imb[name] = round(acc, 4)`	Stocke l'accuracy arrondie à 4 décimales dans le dictionnaire. Ex : results_imb['RF'] = 0.8600

Cellule 9 — Évaluation complète (Table 1 du papier)

Rôle de cette cellule

Calculer toutes les métriques d'évaluation de l'article pour chaque modèle : Accuracy, Precision, Recall, F1-score, Confusion Matrix, AUC-ROC. Reproduit exactement la Table 1 du papier.

Métriques expliquées

`Accuracy`	= (TP+TN)/(TP+TN+FP+FN). % de prédictions correctes parmi toutes les prédictions.

`Precision`	= TP/(TP+FP). Parmi les clients prédits 'churn', combien le sont vraiment ?
`Recall`	= TP/(TP+FN). Parmi les vrais churners, combien ont été détectés ?
`F1-score`	= 2 x (Precision x Recall)/(Precision+Recall). Moyenne harmonique entre precision et recall.
`AUC-ROC`	Area Under ROC Curve. Mesure la capacité du modèle à distinguer churn vs retain. 1.0 = parfait, 0.5 = hasard.

Code complet

```
for name, model in models.items():
    y_pred = model.predict(X_test_ref)
    y_prob = model.predict_proba(X_test_ref)[:, 1]

    print(f'--- {name} ---')
    print(f'Accuracy : {accuracy_score(y_test_ref, y_pred):.4f}')
    print(classification_report(y_test_ref, y_pred))
    print('Confusion Matrix :')
    print(confusion_matrix(y_test_ref, y_pred))
    print(f'AUC-ROC  : {roc_auc_score(y_test_ref, y_prob):.4f}')
```

`model.predict(X_test_ref)`	Prédit la classe (0 ou 1) pour chaque client du test. Retourne un array de 0 et 1.
`model.predict_proba(X_test_ref)[:, 1]`	Retourne les probabilités pour les deux classes. [:, 1] garde seulement la probabilité de la classe 1 (churn). Nécessaire pour AUC-ROC.
`classification_report(y_test_ref, y_pred)`	Affiche precision, recall, f1-score pour chaque classe et les moyennes. Correspond directement à la Table 1 du papier.
`confusion_matrix(y_test_ref, y_pred)`	Matrice 2x2 : [[TN, FP], [FN, TP]]. Montre exactement combien de clients ont été bien/mal classifiés.

Cellule 10 — Figure 3 : Histogramme dataset déséquilibré

Rôle de cette cellule

Reproduire la Figure 3 du papier : histogramme comparant l'accuracy des 4 modèles sur le dataset déséquilibré.

```
plt.figure(figsize=(7, 5))
bars = plt.bar(results_imb.keys(), results_imb.values(),
               color='steelblue', edgecolor='white', width=0.5)
for bar, val in zip(bars, results_imb.values()):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.001,
             f'{val:.2f}', ha='center', va='bottom', fontsize=11)
plt.title('Figure 3 — Accuracy des modèles (Dataset déséquilibré)')
plt.ylim(0.75, 0.92)
plt.show()
```

`plt.bar(results_imb.keys(), results_imb.values())`	Crée un graphique en barres. keys() = noms des modèles (LR, SVM, NN, RF), values() = leurs accuracy.
`for bar, val in zip(bars, results_imb.values()):`	Boucle pour ajouter les valeurs numériques au-dessus de chaque barre.

`plt.ylim(0.75, 0.92)`	Fixe l'axe Y entre 0.75 et 0.92 pour zoomer sur les différences entre modèles (comme dans l'article).

Cellule 11 — Figure 5 : Courbe ROC dataset déséquilibré

Rôle de cette cellule

Reproduire la Figure 5 du papier : courbes ROC des 4 modèles sur le dataset déséquilibré. La courbe la plus proche du coin supérieur gauche indique le meilleur modèle.

```
fig, ax = plt.subplots(figsize=(8, 6))
for name, model in models.items():
    RocCurveDisplay.from_estimator(model, X_test_ref, y_test_ref, ax=ax, name=name)
plt.title('Figure 5 — ROC Curve (Dataset déséquilibré)')
plt.show()
```

`fig, ax = plt.subplots(figsize=(8, 6))`	Crée une figure et un axe matplotlib. On passe 'ax' à RocCurveDisplay pour superposer les 4 courbes sur le même graphique.
`RocCurveDisplay.from_estimator(...)`	Calcule et trace automatiquement la courbe ROC d'un modèle. from_estimator prend directement le modèle entraîné + données test.
`ax=ax, name=name`	ax=ax superpose toutes les courbes sur le même graphique. name=name affiche le nom du modèle dans la légende avec son AUC.

Cellule 12 — SMOTE : Équilibrage du dataset

Rôle de cette cellule

Appliquer SMOTE (Synthetic Minority Over-sampling Technique) pour équilibrer le dataset en créant des exemples synthétiques de la classe minoritaire (churn). Reproduit exactement la Figure 11 du papier. Les instances passent de 10 000 à ~15 924.

Code complet — Figure 11 du papier

```
import pandas as pd
from imblearn.over_sampling import SMOTE

# Initialize SMOTE
smote = SMOTE()

# Resample the data using SMOTE
X_resampled, y_resampled = smote.fit_resample(X, y)
df_resampled = pd.concat([pd.DataFrame(X_resampled, columns=X.columns),
                          pd.Series(y_resampled, name='Exited')], axis=1)

ax = sns.countplot(x=df_resampled['Exited'], palette=['steelblue', 'coral'])
ax.set_title('Number of exited customers after balancing the dataset')
ax.set_xticklabels(['No', 'Yes'])
plt.show()

print(df_resampled.head())
print('Shape après SMOTE :', df_resampled.shape)
```

`from imblearn.over_sampling import SMOTE`	Importe SMOTE depuis imbalanced-learn. SMOTE = Synthetic Minority Over-sampling Technique.
`smote = SMOTE()`	Crée l'objet SMOTE avec paramètres par défaut. Exactement comme dans le code de la Figure 11 du papier.
`smote.fit_resample(X, y)`	Analyse les exemples de la classe minoritaire (churn=1) et génère de nouveaux exemples synthétiques en interpolant entre voisins proches. Retourne X et y augmente.
`pd.concat([X_resampled, y_resampled], axis=1)`	Recompose un DataFrame complet en collant features et cible côte à côte. axis=1 = coller en colonnes.
`ax.set_xticklabels(['No', 'Yes'])`	Reproduit exactement le titre et les labels de la Figure 11 du papier.

Cellule 13 — Division 75/25 dataset équilibré (Figure 12)

Rôle de cette cellule

Reproduire exactement le code de la Figure 12 du papier pour diviser le dataset équilibré. Notez la structure identique au code de l'article avec df.copy().

```
df_aftersplit = df_resampled.copy()
# Assuming 'df' is the DataFrame you want to split
X_bal = df_aftersplit.drop('Exited', axis=1)   # Features
y_bal = df_aftersplit['Exited']                # Target variable

# Split the data into train and test sets
X_train_b, X_test_b, y_train_b, y_test_b = train_test_split(
    X_bal, y_bal, test_size=0.25, random_state=40
)

print('Train set shape:', X_train_b.shape, y_train_b.shape)
print('Test  set shape:', X_test_b.shape,  y_test_b.shape)
```

`df_aftersplit = df_resampled.copy()`	Crée une copie indépendante du DataFrame pour éviter de modifier df_resampled.
`test_size=0.25`	25% test (vs 20% pour les modèles de référence). L'article modifie cette valeur spécifiquement pour ERFA.
`X_train_b, X_test_b, y_train_b, y_test_b`	Suffixe '_b' pour 'balanced'. Variables distinctes du split déséquilibré pour ne pas les mélanger.

Cellule 14 — 4 modèles défaut sur dataset équilibré

Rôle de cette cellule

Entraîner les 4 modèles de référence sur le dataset équilibré pour comparer leurs performances et justifier l'amélioration ERFA. Correspond à la Figure 4 du papier.

```
models_bal = {
    'LR' : LogisticRegression(),
    'SVM': SVC(probability=True),
    'NN' : MLPClassifier(max_iter=1000),
    'RF' : RandomForestClassifier()
```

```
}

results_bal = {}
for name, model in models_bal.items():
    model.fit(X_train_b, y_train_b)
    acc = model.score(X_test_b, y_test_b)
    results_bal[name] = round(acc, 4)
    print(f'{name}: {acc:.4f}')
```

`models_bal = { ... }`	Nouveau dictionnaire séparé de 'models'. Obligatoire car on entraîne sur X_train_b/y_train_b (équilibré) et non X_train_ref.
`model.fit(X_train_b, y_train_b)`	Entraînement sur le dataset équilibré par SMOTE. Les modèles voient maintenant autant d'exemples churn que retain.
`model.score(X_test_b, y_test_b)`	Évaluation sur le test équilibré. Résultats attendus du papier : LR=70%, SVM=78%, NN=89%, RF=89%.

Cellule 15 — Figure 7 : Division ERFA (75/25 déséquilibré)

Rôle de cette cellule

Reproduit exactement le code de la Figure 7 du papier. C'est la division train/test spécifique à ERFA sur le dataset déséquilibré original. On ré-applique normalisation et sélection de features sur cette nouvelle copie.

```
df_aftersplit = df.copy()
# Assuming 'df' is the DataFrame you want to split
X_erfa = df_aftersplit.drop('Exited', axis=1)  # Features
y_erfa = df_aftersplit['Exited']               # Target variable

# Ré-appliquer normalisation
X_erfa_scaled = scaler.fit_transform(X_erfa)
X_erfa = pd.DataFrame(X_erfa_scaled, columns=X_erfa.columns)
X_erfa = X_erfa[selected_features]

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X_erfa, y_erfa, test_size=0.25, random_state=40
)
print('Train set shape:', X_train.shape, y_train.shape)
print('Test  set shape:', X_test.shape,  y_test.shape)
```

`df_aftersplit = df.copy()`	Travaille sur une copie fraîche du df original (déséquilibré). Identique à la Figure 7 du papier.
`X_erfa = X_erfa[selected_features]`	Applique la même sélection de features (corrélation >= 10%) définie à la cellule 5.
`test_size=0.25`	ERFA utilise 75% train / 25% test. Différent de 80/20 des modèles de référence. C'est une des modifications clés de l'article.

Cellule 16 — Figure 8 : ERFA dataset déséquilibré

Rôle de cette cellule

C'est le cœur du papier. Reproduit exactement le code de la Figure 8 : RandomForestClassifier avec les 4 hyperparamètres modifiés. C'est ici que l'amélioration ERFA est implémentée.

Les 4 hyperparamètres ERFA (Mathematical Model du papier)
n_estimators=30 (30 arbres au lieu de 100) | criterion='gini' (Index Gini) | max_features=4 (fs=sqrt(16)=4) | max_depth=26 (profondeur maximale des arbres)

Code complet — Figure 8 exacte

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics  import classification_report, confusion_matrix

# Create a Random Forest model
rf = RandomForestClassifier(n_estimators=30, criterion='gini', max_features=4, max_depth=26)
#rf = RandomForestClassifier()

# Train the model on the training data
rf.fit(X_train, y_train)

# Make predictions on the test data
y_pred = rf.predict(X_test)

# Evaluate the model performance
accuracy = rf.score(X_test, y_test)

# Print the accuracy
print('Accuracy:', accuracy)
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)

# Print the confusion matrix
print(cm)
```

`n_estimators=30`	Nombre d'arbres de décision dans la forêt. L'article fixe 30 (vs 100 par défaut) pour être plus efficace en temps de calcul.
`criterion='gini'`	Critère de split des noeuds = Index Gini. C'est la mesure utilisée dans le modèle mathématique du papier pour calculer l'importance des features.
`max_features=4`	Nombre de features considérées pour chaque split. Formule du papier : fs = sqrt(16) = 4. (16 = nombre de features utiles dans leur dataset).
`max_depth=26`	Profondeur maximale de chaque arbre. L'article justifie : 'for a deep analysis of customer churn features in all cases, we assume Tree Depth = 26'.
`#rf = RandomForestClassifier()`	Ligne commentée dans le code original de la Figure 8 du papier. Montre le RF défaut pour comparaison.
`rf.fit(X_train, y_train)`	Entraîne ERFA sur 75% des données. Les 30 arbres sont construits en parallèle avec bootstrap sampling.

`accuracy = rf.score(X_test, y_test)`	Calcule l'accuracy sur 25% test. Attendu : ~87% (vs 86% RF défaut).
`print('Accuracy:', accuracy)`	Affiche l'accuracy — exactement comme dans la Figure 8 du papier.

Cellule 17 — Figure 10 : Courbe ROC ERFA déséquilibré

Rôle de cette cellule

Reproduire la Figure 10 du papier : courbe ROC de ERFA sur le dataset déséquilibré. AUC attendu : 0.95.

```
from sklearn.metrics import RocCurveDisplay

fig, ax = plt.subplots(figsize=(8, 6))
RocCurveDisplay.from_estimator(rf, X_test, y_test, ax=ax,
    name=f'Random Forest (AUC = {roc_auc_score(y_test, rf.predict_proba(X_test)[:,1]):.2f})')
plt.title('Figure 10 — ROC Curve — ERFA (Dataset déséquilibré)')
plt.show()
```

`rf.predict_proba(X_test)[:,1]`	Probabilités de churn pour chaque client. [:,1] = colonne de la classe 1. Nécessaire pour tracer la courbe ROC.
`f'Random Forest (AUC = ...)'`	Formate le nom de la courbe avec l'AUC calculée dynamiquement, comme dans la Figure 10 du papier.

Cellule 18 — Figure 13 : ERFA dataset équilibré

Rôle de cette cellule

Même code que la Figure 8 mais appliqué au dataset équilibré (SMOTE). Reproduit exactement la Figure 13 du papier. AUC attendu : 0.96. Accuracy attendue : 91%.

```
from sklearn.ensemble import RandomForestClassifier

# Create a Random Forest model
rf = RandomForestClassifier(n_estimators=30, criterion='gini', max_features=4, max_depth=26)
#rf = RandomForestClassifier()

# Train the model on the training data
rf.fit(X_train_b, y_train_b)

# Make predictions on the test data
y_pred = rf.predict(X_test_b)

# Evaluate the model performance
accuracy = rf.score(X_test_b, y_test_b)

print('Accuracy:', accuracy)
from sklearn.metrics import classification_report
print(classification_report(y_test_b, y_pred))

cm = confusion_matrix(y_test_b, y_pred)
print(cm)
```

`rf.fit(X_train_b, y_train_b)`	Même ERFA (mêmes hyperparamètres) mais entraîné sur X_train_b (équilibré par SMOTE). Le modèle voit autant de churners que de non-churners.
`rf.predict(X_test_b)`	Prédictions sur le test équilibré. Attendu : meilleur recall sur la classe churn comparé au cas déséquilibré.
`accuracy = rf.score(X_test_b, y_test_b)`	Accuracy attendue : 91% selon le papier. Amélioration significative grâce à SMOTE + hyperparamètres ERFA.

Cellule 19 — Visualisation finale comparative

Rôle de cette cellule

Reproduire les Figures 3 et 4 du papier côte à côte avec les résultats réels de tous les modèles, y compris ERFA. C'est la visualisation de conclusion du papier.

```
# Ajouter ERFA aux résultats
erfa_imb_acc = RandomForestClassifier(
    n_estimators=30, criterion='gini', max_features=4, max_depth=26
)
erfa_imb_acc.fit(X_train, y_train)
results_imb['ERFA'] = round(erfa_imb_acc.score(X_test, y_test), 4)

erfa_bal_acc = RandomForestClassifier(
    n_estimators=30, criterion='gini', max_features=4, max_depth=26
)
erfa_bal_acc.fit(X_train_b, y_train_b)
results_bal['ERFA'] = round(erfa_bal_acc.score(X_test_b, y_test_b), 4)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
# ... barres + labels ...
plt.show()
```

`results_imb['ERFA'] = ...`	Ajouter ERFA au dictionnaire des résultats déséquilibrés pour l'inclure dans la comparaison finale.
`results_bal['ERFA'] = ...`	Idem pour le cas équilibré.
`plt.subplots(1, 2, figsize=(14, 6))`	Crée 2 graphiques côte à côte (1 ligne, 2 colonnes). Reproduit la présentation des Figures 3 et 4 du papier.

Cellule 20 — Feature Importance (Figure 2 du papier)

Rôle de cette cellule

Visualiser l'importance de chaque feature dans ERFA. Reproduit la Figure 2 du papier qui montre les 'Top Predictive Features'. Basé sur la réduction moyenne de l'impureté Gini dans les 30 arbres.

```
importances = pd.Series(
    erfa_imb_acc.feature_importances_,
    index=X_train.columns
).sort_values(ascending=False)

plt.figure(figsize=(12, 5))
importances.plot(kind='bar', color='steelblue', edgecolor='white')
plt.title('Figure 2 — Top Predictive Features (ERFA Feature Importance)')
plt.ylabel('Importance')
```

```
plt.xlabel('Features')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

print('Top features :')
print(importances)
```

`erfa_imb_acc.feature_importances_`	Attributs sklearn qui donne l'importance de chaque feature après entraînement. Calculé comme la réduction moyenne de l'impureté Gini sur les 30 arbres.
`pd.Series(..., index=X_train.columns)`	Créer une Séries pandas avec les noms de colonnes comme index pour que le graphique affiche les noms des features.
`.sort_values(ascending=False)`	Trie de la plus importante à la moins importante. Les premières barres = features qui contribuent le plus à prédire le churn.
`plt.xticks(rotation=45, ha='right')`	Rotation des labels de l'axe X pour qu'ils soient lisibles sans se chevaucher.

Résumé — Paramètres ERFA vs RF défaut

Paramètre	RF défaut	ERFA	Pourquoi ce choix
n_estimators	100	30	Efficacité — 30 arbres suffisent selon le papier
criterion	gini	gini	Inchangé — Gini est le critère du modèle mathématique
max_features	sqrt auto	4 (=√16)	fs = √(nb_features_utiles) selon la formule du papier
max_depth	None (infini)	26	Analyse profonde des features churn
test_size	0.20 (80/20)	0.25 (75/25)	Donner plus de données au modèle
random_state	None	40	Reproductibilité — valeur du papier